

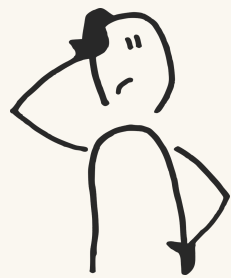
APRESENTAÇÃO

Projeto de Inteligência Artificial

GRUPO

Amanda Lucas (202400455)
Rafael Trivelatto (202408221)
Simão Martins (202404938)

Project Overview



A ideia

Construir uma game engine do jogo PopOut em que se fazem disponíveis 3 modos de jogo:

- 1) Humano vs Humano
- 2) Humano vs Computador
- 3) Computador vs Computador



O processo

Começamos por entender o jogo e as suas nuances em relação ao connectFour tradicional.

Depois de implementar as regras e uma interface para jogadores humanos, aplicamos a teoria por trás da MCTS ao nosso jogo. Por fim, treinamos a árvore de decisão em dados sintéticos.



O resultado

Como esperado, a IA baseada na MCTS acabou praticamente imbatível para um jogador casual, e a árvore de decisão mostra um bom desempenho em geral, embora não seja tão flexível.

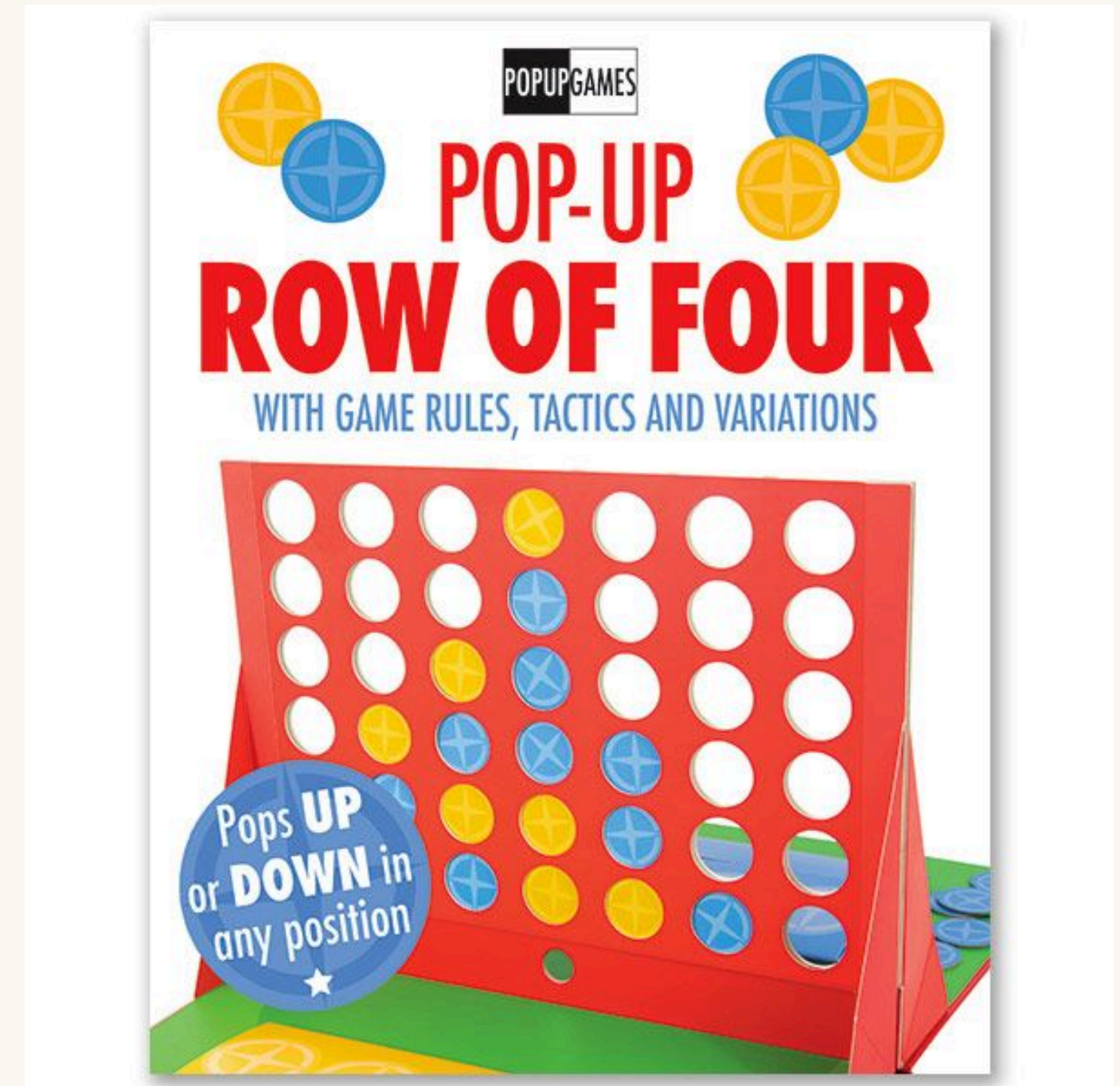
O jogo

PopOut

É uma variação do jogo “connect four”. É jogado por dois agentes, com peças de cores diferentes, numa grelha 6x7. A cada jogada, um jogador pode jogar uma peça numa coluna ou remover uma peça sua que esteja na base de uma coluna.

Regras especiais

- Se um pop levar a um “four in line” para cada jogador, vence o jogador que fez o pop;
- Se a grelha estiver cheia, o jogador pode declarar empate ou continuar o jogo com um pop;
- Se um estado do jogo for repetido três vezes, o jogo termina e é considerado um empate.



Estrutura da game engine

1

Criamos uma classe “Board” com a gestão do tabuleiro, incluindo verificação de vitórias.

2

Criamos uma classe “Player” que serve como template para os vários tipos de jogador (Human, MCTS Clássico, MCTS Adptado). Inclui um método que determina os movimentos legais numa dada configuração do jogo.

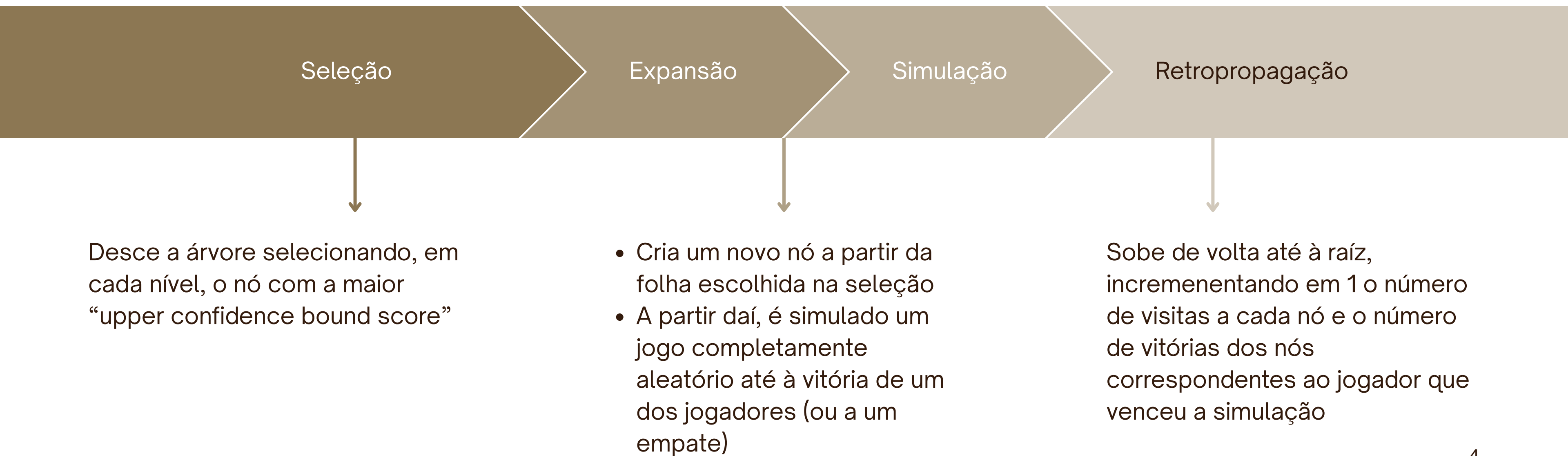
3

Implementamos “HumanPlayer” e “MCTSPlayer” como subclasse de “Player”, para facilitar a interação do jogador humano com a game engine, tal como o bom funcionamento quando um player é assumido pelo algoritmo MCTS.

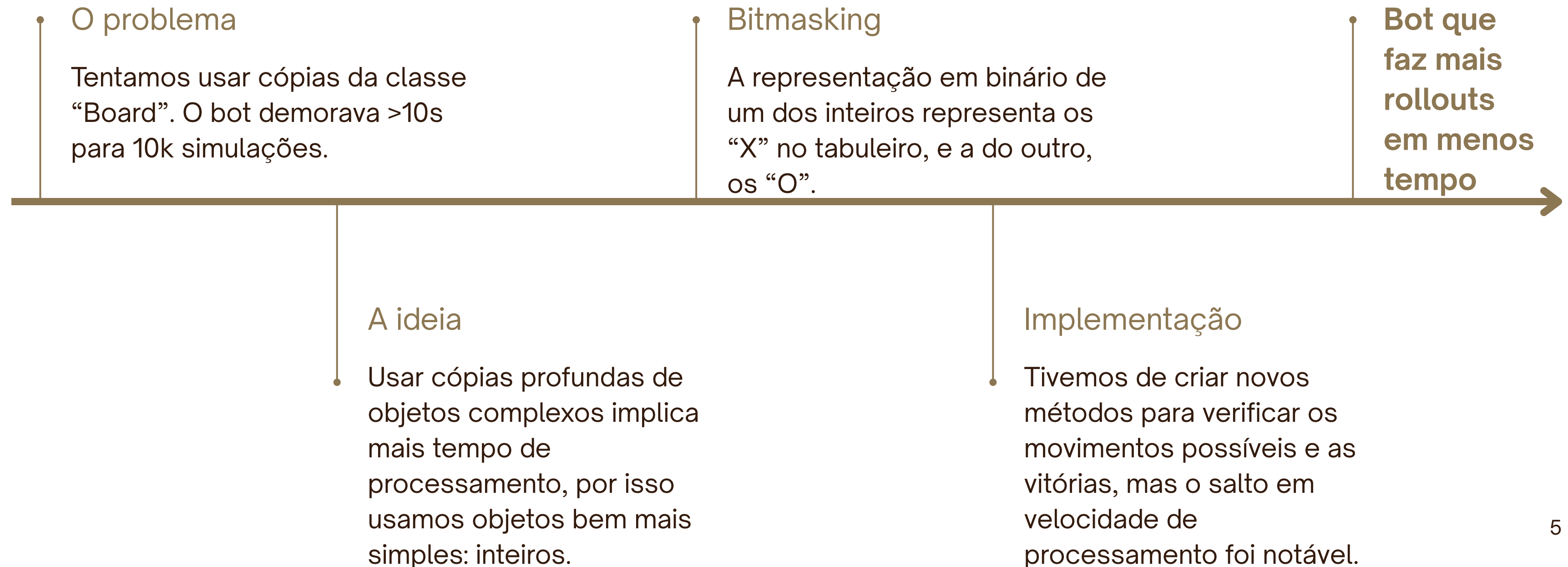
Nota: usamos arrays de numpy para representar o tabuleiro

Isto acelerou significativamente a verificação de vitórias. Mesmo assim, para os rollouts da nossa MCTS, de modo a alcançar o máximo de simulações no mínimo de tempo possível, usamos uma representação diferente.

Implementação do MCTS



Simulações focadas em performance



Heurísticas Implementadas:

Progressive Bias:

Guia as jogadas para o centro do tabuleiro.

First Play Urgency:

UCB dinamico para nós não explorados.

Smart RollOut:

Detecção de vitórias/derrotas para a próxima jogada.

Outras Mudanças:

Alteração da constante C:

Alteramos a constante C para um valor menor para observar o comportamento mais conservador do algoritmo

Limitação no número de filhos:

Ao limitar o numero de filhos que um nó pode ter limitamos suas opções em jogadas mais conservadoras

Conclusões



Estagnação de desempenho

Mesmo com heurísticas bem aplicadas notamos uma estagnação no desempenho ao fazer simulações com elevado numero de iterações



Conclusão:

As heurísticas além de dar uma noção de jogo ao algoritmo também ajuda a pouoar recursos, sento mais notavel com menor número de iterações

```
=====
✅ Placar Final - Buffado (6 filhos) - 8000 iter (60.12 min):
🏆 Vitórias do Buffado: 40/100
💀 Vitórias do Clássico: 58/100
👑 Empates: 2/100
🎯 WinRate: 40.0%
🔄 Jogos: 100
=====
```

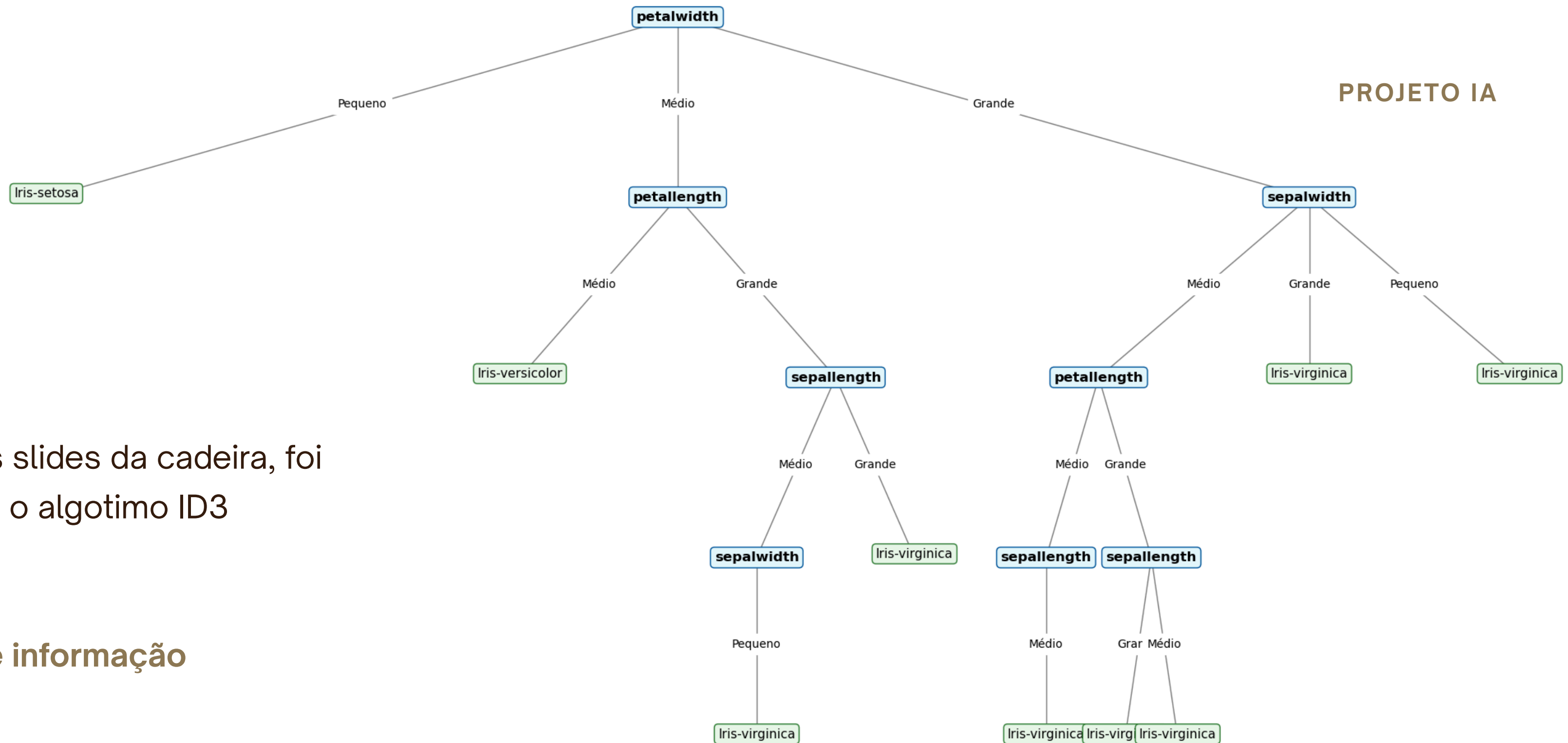

DATASET

ID3

Com base nos slides da cadeira, foi implementado o algoritmo ID3

- Entropia
- Ganho de informação
- my_ID3
- Classificador
- Árvore

PROJETO IA



Dataset MCTS

Afim de usar os dados de melhor qualidade de jogada, dividimos o dataset em 2 arquivos, um com os dados de jogo do MCTS vencedor, e outro do perdodor do jogo. Para treinar a árvore utilizamos apenas os dados do vencedor.

[illegible]